


```
// build a BST
public static Node insert(Node root, int val){
    if(root == null){
        root = new Node(val);
        return root;
    }
    if(val < root.data){
        root.left = insert(root.left, val);
    }else{
        root.right = insert(root.right, val);
    }

    return root;
}
```

```
int[] values = {8, 5, 3, 1, 4, 6, 7, 10, 11, 14};
Node root = null;

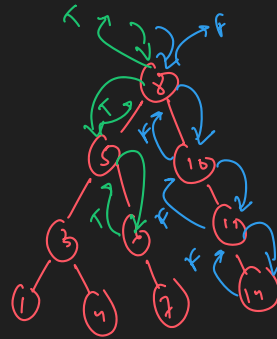
for(int i=0; i<values.length; i++){
    root = insert(root, values[i]);
}

inOrder(root);
System.out.println();
```

Q Search in a BST

$k = 7$

```
public static boolean searchInBST(Node root, int target){
    if(root == null){
        return false;
    }
    if(target < root.data){
        return searchInBST(root.left, target);
    }else if(root.data < target){
        return searchInBST(root.right, target);
    }else{
        System.out.println(root.data);
        return true;
    }
}
```

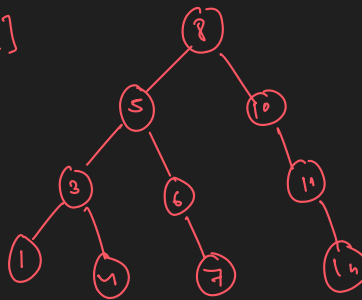


$(k) \rightarrow f$

Q Print a Range $[k_1 < k_2]$

$k_1 = 5$ and $k_2 = 12$

o/p: 5 6 7 8 10 11



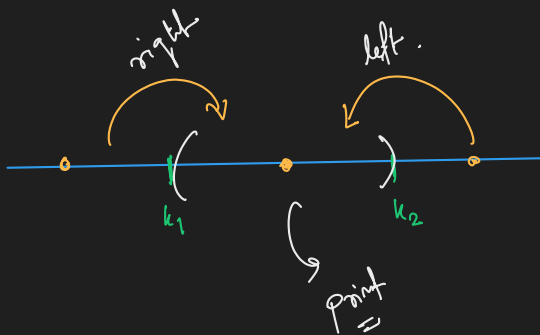
① $k_1 \leq \text{root.data} \leq k_2 \rightarrow \text{Print}$

② $\text{root.data} > k_2$

$\rightarrow$ left sub tree

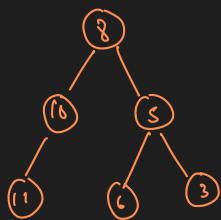
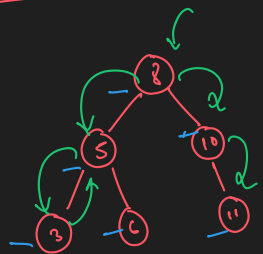
③ $\text{root.data} < k_1$

$\rightarrow$ right sub tree



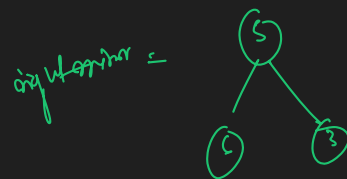
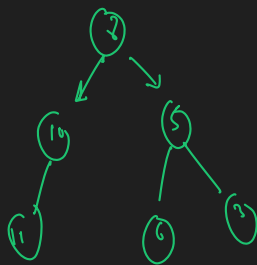
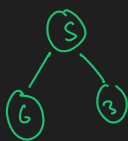
```
// Print in Range k1 and k2
public static void printInRange(Node root, int k1, int k2){
    if(root == null){
        return;
    }
    if(k1 <= root.data && root.data <= k2){
        printInRange(root.left, k1, k2);
        System.out.print(root.data+" ");
        printInRange(root.right, k1, k2);
    }else if(root.data > k2){
        printInRange(root.left, k1, k2);
    }else{ // root.data < k1
        printInRange(root.right, k1, k2);
    }
}
```

Q Mirror a BST



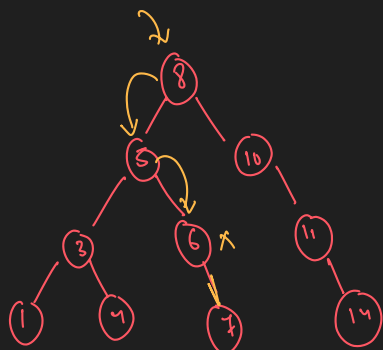
pre order = 8 10 11 5 6 3
in order = 11 10 8 6 5 3

left → left mirror
right → right mirror = 3



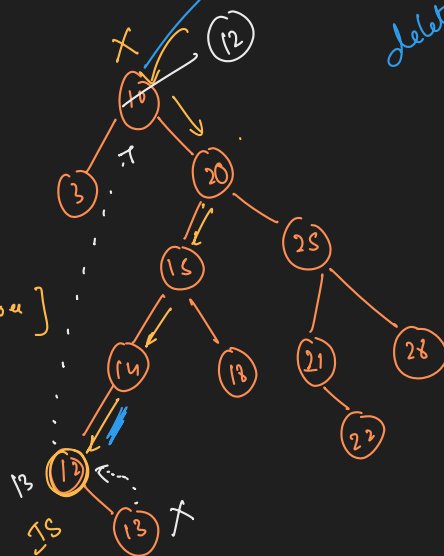
Imp Delete a node

- (i) Delete leaf node 7
- (ii) Delete a node which has exactly one child 10



- (iii) Delete a node which has two children 10

1. find in order predecessor [right most node of left subtree]
2. find in order successor [left most node of right subtree]



delete (is, is)

3, 10, 12, 13, 14, 15, 18, 20, 21, 22, 25, 28

Property → IS / IP
It has at most one child

```
// delete a Node form a BST
public static Node delete(Node root, int val){
    // find the value to be deleted
    if(root.data > val){
        root.left = delete(root.left, val);
    }else if(root.data < val){
        root.right = delete(root.right, val);
    }else{ // root.data = val
        // leaf Node
        if(root.left == null && root.right == null){
            return null;
        }

        // One Child
        if(root.left == null){
            return root.right;
        }else if(root.right == null){
            return root.left;
        }

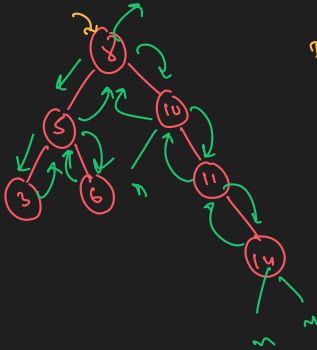
        // Two Children
        Node IS = inOrderSuccessor(root.right);
        root.data = IS.data;
        root.right = delete(root.right, IS.data);
    }

    return root;
}
```

```
public static Node inOrderSuccessor(Node root){
    while (root.left != null) {
        root = root.left;
    }
    return root;
}
```

Q

Root to leaf Path



Path 1: 8 | 5 | 3

Path 2: 8 | 5 | 6

Path 3: 8 | 10 | 11 | 14

AL Path = 8 | 10 |

```
// root to all path
public static void allPath(Node root, ArrayList<Integer> path){
    if(root == null){
        return;
    }

    path.add(root.data);

    // leaf node
    if(root.left == null && root.right == null){
        System.out.println(path);
    }
    allPath(root.left, path);
    allPath(root.right, path);

    path.remove(path.size()-1);
}
```